North-West University (NWU)
Faculty of Natural Science and Agriculture

ITRI626 – Deep learning for image classification · Practical project report

Recognising electronic components in web photographs

A leakage-free comparison of a scratch CNN, a fine-tuned ConvNeXt and a frozen SigLIP Vision Transformer

Group member	Student number
M van Niekerk	47817909
D Schoeman	45616779
M Adriaanzen	43054226
S Klopper	45757895

Lecturer: Prof. Absalom Ezugwu · Facilitator: Mr. Bernard Swanepoel · 20 October 2026, revised 4 November 2026

Abstract. We classify web photographs of electronic components into 20 classes (resistor, capacitor, relay, microcontroller board and 16 others) using the public Kaggle dataset *Electronic Component Recognizer*. After exact duplicates and 13 groups with contradictory labels are removed, 2,930 images remain; near-duplicates are grouped so that no image or re-encoded copy crosses a split boundary, giving a stratified split (seed 42) of 2,010 training, 502 validation and 418 test images. Three model families are trained on this split and scored once on the same test set: (A) a VGG-style CNN trained from scratch, (B) ConvNeXt-Tiny fine-tuned from ImageNet-22k weights and (C) a frozen SigLIP So400m Vision Transformer with a trained linear head. Test accuracy is 0.400, 0.636 and 0.823 (macro-F1 0.392, 0.628, 0.817); the 95% bootstrap intervals do not overlap and McNemar tests reject equal accuracy for every pair. Pretraining accounts for the ranking: ImageNet fine-tuning adds 23.7 percentage points over the scratch baseline, and web-scale image–text pretraining a further 18.7. Most remaining errors involve look-alike parts or images that do not show the labelled component. We recommend model C, and ConvNeXt-T where inference must be fast or run on a CPU.

1 Introduction and problem motivation

Problem. Given one photograph, assign it to exactly one of 20 classes of electronic components and modules (Table 1). This is single-label, multiclass image classification of web photographs of arbitrary origin: product shots, close-ups, cluttered workbenches, renders and some diagrams.

Motivation. Recognising a part from a photograph helps to sort mixed parts bins and take inventory, to teach students to tell a diode from a resistor, and as the recognition stage of lab-assistant tools. Earlier work on electronic-component classification fine-tunes ImageNet networks [1] or designs dedicated CNNs for varied backgrounds [2]; inspection of assembled circuit boards is surveyed by Ling and Isa [3]. The task is also a good test of transfer learning. There are only about 100 training images per class, the data are noisy, and several classes look alike (resistor, diode and fuse are small axial bodies with leads; inductor coil and transformer are both windings).

Aim and scope. We compare three model families, one trained from scratch and two transfer-learning strategies, on one leakage-free train/validation/test split, evaluate all of them on the same held-out test set, explain their differences and recommend one. The scope is whole-image classification of the dataset's 20 classes; detecting several components in one image is out of scope.

Research question. Which way of using a neural network (training from scratch, fine-tuning ImageNet features, or a linear classifier on frozen web-scale image–text features) classifies these noisy web photographs best, and at what computational cost?

Main findings.

- Model C is the most accurate (accuracy 0.823, macro-F1 0.817), ahead of model B (0.636, 0.628) and model A (0.400, 0.392); every pairwise difference is significant (Section 5).
- The ranking follows pretraining: ImageNet fine-tuning adds 23.7 percentage points over training from scratch, and image–text pretraining a further 18.7 (Section 6.1).
- Most remaining errors are look-alike parts or images that do not show the labelled part; data quality, not class imbalance, limits accuracy (Section 6.2).
- We recommend model C, or ConvNeXt-T where inference must be fast or run on a CPU (Section 7).

Target classes. We keep the dataset's 20 classes because they cover the parts on a typical electronics workbench, are defined by a public source (so the work is reproducible), are roughly balanced (Section 2.4), and include genuinely confusable groups that make a model comparison informative.

2 Dataset: source, classes, preparation and ethics

2.1 Source, licence and usage conditions

The images come from the Kaggle dataset *Electronic Component Recognizer* by the user `mdfaisalahmed025` [4], `https://www.kaggle.com/datasets/mdfaisalahmed025/electronic-component-recognizer`, released under the **MIT licence** (use, copying, modification and redistribution with the copyright notice, without warranty). The author collected the images with DuckDuckGo image search, one query per class, so **the labels are the search terms**. The dataset card states 3,661 images, but our download contains 3,530 files; we could not determine the reason (the card may describe an earlier version), and all counts below refer to the files we received. The MIT licence covers the compilation, not the copyright of the individual web images, which stays with their owners. We therefore use the images only for this non-commercial course project, acknowledge the source, and do not commit the images to the project's Git repository. The download also ships a random train/validation/test assignment of the files; we do not use it, because it places copies of the same image on both sides (Section 2.5).

2.2 Class definitions and counts

Table 1 defines each class and gives its image count at every cleaning stage and in every split; Figure 1 shows four random training images per class.

2.3 Cleaning, label checking and corrections

All cleaning is done by `tools/build_dataset_v2.py`, which writes the manifest `data/v2/manifest.json` and the audit `data/v2/leakage_report.json`; Table 2 summarises the counts.

1. **Decoding.** Every file was decoded and stored as an RGB JPEG (the unique files were 1,486 JPEG, 787 WebP, 402 PNG, 245 AVIF, 20 GIF and 3 MPO files; 345 transparent images were composited on white). No file was corrupt.
2. **Exact duplicates.** Files with identical SHA-256 hashes were collapsed into one image (587 redundant copies in 533 groups). 13 groups (27 files) held the same image under two class folders, e.g. *fuse* and *relay*; their labels contradict each other, so they were **removed**, leaving 2,930 images.
3. **Near-duplicates.** Re-encoded or resized copies were linked when their perceptual-hash distance (the Hamming distance between pHash codes) was at most 10 and the cosine similarity of their CLIP ViT-B/32 [5] embeddings at least 0.85, or when the cosine alone was at least 0.92. This gave 91 clusters with 234 images (sizes 2 to 29), which are kept but always assigned to the same split (Section 2.5).
4. **Irrelevant or uncertain samples.** A CLIP zero-shot screen compares each image with the 20 class prompts and 9 distractor concepts (person, car, food, logo, text screenshot, cartoon or game art, circuit schematic, website screenshot, collage) and flags it when the distractor probability mass exceeds 0.25. It flagged 816 images (27.8%), concentrated in classes whose names are common words: 121 of 177 *switch* images (mostly Nintendo Switch consoles) and

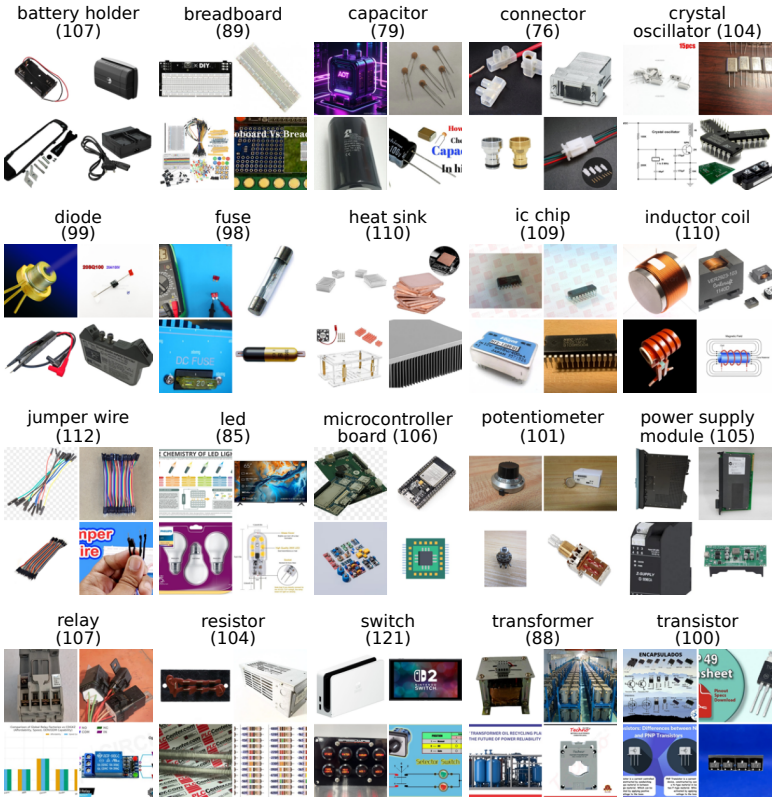


Figure 1: Four random training images of every class (in brackets: training images of the class). The mix of product photos, close-ups, renders, diagrams and off-topic pictures (e.g. the game console under *switch*, the infographic under *led*) is typical of the dataset.

Table 1: The 20 classes, their image counts at each cleaning stage and in each split (`data/v2/splits/split_summary.csv`).

		Cleaning				Split			
Class	Definition	Raw	Uniq.	Excl.	Final	Train	Val	Test	Flag.
battery holder	cradle or box holding cylindrical or coin cells	220	155	0	155	107	26	22	31
breadboard	solderless prototyping board with spring-clip holes	141	135	4	131	89	23	19	22
capacitor	two-terminal charge-storage part (electrolytic, ceramic, film)	121	117	1	116	79	20	17	16
connector	plug, socket or pin header joining cables or boards	114	110	0	110	76	18	16	11
crystal oscillator	quartz crystal or oscillator can / SMD clock package	232	152	1	151	104	26	21	50
diode	two-terminal rectifier, usually a banded axial body	151	148	3	145	99	25	21	56
fuse	protective glass, ceramic or blade cartridge	148	146	4	142	98	24	20	46
heat sink	finned metal block that dissipates heat	164	160	0	160	110	27	23	19
ic chip	packaged integrated circuit (DIP, SOIC, QFP)	166	157	0	157	109	26	22	15
inductor coil	wire wound on a core (toroid, choke, coil)	244	162	1	161	110	28	23	26
jumper wire	insulated lead with pin or socket ends	250	164	1	163	112	28	23	62
led	light-emitting diode (dome package, strip)	174	125	1	124	85	21	18	55
microcontroller board	development board with an MCU (Arduino, ESP32)	155	154	0	154	106	26	22	20
potentiometer	variable resistor with shaft or trimmer screw	161	147	0	147	101	25	21	14
power supply module	board-level AC/DC or DC/DC converter	251	156	2	154	105	27	22	23
relay	electrically operated switch (boxed coil and contacts)	167	160	4	156	107	27	22	61
resistor	fixed two-terminal resistor, colour bands or SMD	171	155	2	153	104	27	22	43
switch	manual switch (toggle, push-button, slide, DIP)	188	177	0	177	121	31	25	121
transformer	coupled windings on a laminated core	147	129	1	128	88	22	18	64
transistor	three-terminal semiconductor (TO-92, TO-220)	165	148	2	146	100	25	21	61
Total		3,530	2,957	27	2,930	2,010	502	418	816

Raw: files in the Kaggle download. Uniq.: unique images per class folder after collapsing exact duplicates (an image saved under two folders counts in both). Excl.: files in exact-duplicate groups with conflicting labels (removed). Final: images kept. Train/Val/Test: the fixed split. Flag.: images flagged as possibly off-topic (out of distribution) by the CLIP screen (kept).

64 of 128 *transformer* images (toy robots, cartoons). Flagged images are **kept and reported separately**: in earlier 5-fold cross-validation, dropping them from training lowered the accuracy of a linear classifier on frozen SigLIP features (a *linear probe*, Section 3.3) from 0.825 to 0.792 overall and from 0.861 to 0.852 on unflagged images.

5. **Label errors.** A confident-learning audit [6] on out-of-fold probe probabilities listed 69 images whose given label is improbable while another class is highly probable.

How labels were obtained, checked and corrected. Labels come from the search query. They were checked automatically (conflicting duplicates, CLIP screen, confident-learning audit) and by inspecting the review sheets in `data/v2/review/`. The only correction applied was the removal of the 13 conflicting groups; we did not relabel images, which would need a manual review of all 2,930 images by more than one annotator. The test labels therefore contain the same noise as the training labels, which caps every model's measurable accuracy.

Table 2: Cleaning pipeline (`data/v2/leakage_report.json`); indented rows break down the row above.

Step	Count	Action
Raw image files in the Kaggle download	3,530	–
Unique files (distinct SHA-256 of the bytes)	2,943	–
redundant exact copies (533 duplicate groups)	587	collapsed to one
corrupt or undecodable files	0	–
duplicate groups with conflicting labels (27 files)	13	removed
Final images	2,930	kept
images in near-duplicate clusters (91 clusters, size 2–29)	234	kept, grouped
split units: clusters incl. singletons (groups, not images)	2,787	–
images flagged as possibly off-topic (CLIP screen)	816	kept, flagged

2.4 Class imbalance

After cleaning, the largest class (*switch*, 177 images) is 1.61 times the smallest (*connector*, 110); training classes have 76–121 images and test classes 16–25 (Table 1, Figure B.1 in Appendix B). Because the imbalance is mild we used **no class weights and no re-sampling**. Instead the split is stratified by class, early stopping and model selection use validation macro-F1 (every class counts equally), and we report macro averages and weighted F1.

2.5 Train/validation/test split and leakage prevention

The unit of splitting is the *duplicate cluster*: a near-duplicate cluster or a single image without near-duplicates (2,787 clusters). We split in two stages and always keep a cluster whole. First, `StratifiedGroupKFold` (scikit-learn [7]; 7 folds, shuffled, **seed 42**) partitions the clusters, and the fold whose class distribution is closest (total-variation distance) to the overall one becomes the **test** set (418 images), which is held out ("sealed") until the final evaluation. Second, the remaining 2,512 development images are divided by the same method into five grouped folds: fold 0 is the **validation** set (502) and folds 1–4 form the **training** set (2,010), i.e. 68.6/17.1/14.3%, close to the 70/15/15 split suggested in the brief. The class distribution of each split is in Table 1. `tools/export_splits.py` writes the split from the manifest (fingerprint `431315e87d2612cb`) to `data/v2/splits/{train,val,test}.csv` and, with `--check`, verifies the files; the notebook and `make_tables.py` assert that no image and no cluster occurs in two splits. Grouping matters: in the dataset's original random file split (counted over the raw file entries), 128 of 522 validation and 115 of 523 test entries are byte-identical copies of a training file, which inflates scores [8, 9] (Section 6.4). The test set was used once per model, after all choices had been fixed on validation data.

2.6 Ethical, privacy and licensing considerations

The task uses no health or personal records, and no person is a subject of the classification. As a web scrape, the data contain incidental people (the CLIP screen names "person" as the top distractor for 83 images) and trademarks (game consoles, brand logos). We identify no one; such images appear in our figures only as small thumbnails from the public dataset. Use follows Section 2.1: the source is acknowledged, the use is non-commercial and educational, and the images are not redistributed in the repository (they are rebuilt from the Kaggle download, Appendix D). The 407 photographs of the supplementary web holdout (Section 6.4) are from Wikimedia Commons under public-domain or Creative-Commons licences; author, licence and source of each are listed in `data/web_holdout_v1/ATTRIBUTION.md`. Pretrained weights come from their official `timm` releases [10]. The main risk is over-trust in a wrong label, which is why we report confidence and a reject option. Web search results may also over-represent clean product shots, so performance on cluttered workbench photographs can be lower (cf. the Wikimedia holdout in Section 6.4).

3 Methodology, selected models and transfer learning

Figure 2 shows the workflow. The same split files feed all three models; each is selected on the validation set only and scored once on the test set.

3.1 Model A: VGG-style CNN trained from scratch (baseline)

A VGG-style network [11] written for this project (`src/compclass/models/scratch_cnn.py`): a stride-2 stem convolution, four stages of two 3×3 convolution–BatchNorm [12]–ReLU blocks (32, 64, 128, 256 channels) with max-pooling between stages, global average pooling, dropout 0.3 and a linear classifier; 1,187,636 parameters with He initialisation [13] and **no pretrained weights**. It shows what can be learned from about 2,000 images alone.

3.2 Model B: ConvNeXt-Tiny, fine-tuned (transfer learning by fine-tuning)

ConvNeXt [14] modernises the ResNet design [15] with 7×7 depthwise convolutions, inverted bottlenecks, LayerNorm and GELU. We use the Tiny variant (27,835,508 parameters) with the `timm` weights `convnext_tiny.fb_in22k_ft_in1k`, pretrained on ImageNet-22k and fine-tuned on ImageNet-1k [16]. The 1000-class head is replaced by a new 20-class linear layer, which is trained alone in the first epoch so that its large initial gradients do not disturb the pretrained features [17]. Afterwards all layers are trained, with layer-wise learning-rate decay 0.8 so that early, generic layers change less than late, task-specific ones. Fine-tuning ImageNet networks is the standard transfer recipe for small datasets [18, 19].

3.3 Model C: SigLIP So400m ViT, frozen, with a linear head (transfer learning by feature extraction)

The Vision Transformer [20] splits an image into patches and processes the patch tokens with self-attention. SigLIP [21] trains a ViT image encoder and a text encoder on WebLI (a web-scale image–text collection) with a pairwise sigmoid loss. We use its shape-optimised So400m/14 encoder at 384×384 pixels (27 blocks, width 1152, patch size 14; `timm` weights `vit_so400m_patch14_siglip_384.webli`). **The backbone is frozen.** Each image is encoded once into its attention-pooled 1152-dimensional embedding; the features are standardised with the training-split mean and standard deviation, and only one `nn.Linear(1152, 20)` layer (23,060 parameters) with softmax output is trained with cross-entropy and AdamW, i.e. the linear-probe protocol used for image–text models [5].

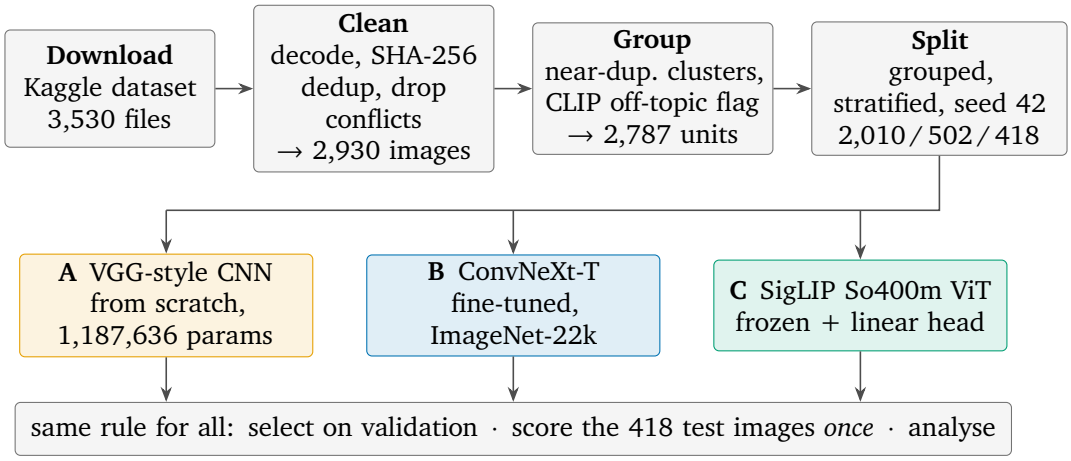


Figure 2: Workflow. Data preparation (top) is shared; the three models (middle, in the colours used in every figure) use the same split files and selection rule and are evaluated on the same 418 test images.

3.4 Rationale for the models and evaluation protocol

The brief lists a small custom CNN, ConvNeXt and the Vision Transformer as separate suitable families; our models take one from each (plain CNN, modernised CNN, Vision Transformer) and span the transfer-learning spectrum: no transfer (A), full fine-tuning of supervised ImageNet features (B) and frozen features from web-scale image–text pretraining (C). Between B and C, architecture and pretraining data change together, so the comparison measures *model plus pretraining* as available to a practitioner; the supplementary benchmark (Section 6.4) separates the two factors.

Evaluation. On the test set we compute per-class precision, recall and F1, their macro averages (all classes equal) and weighted averages (by support), accuracy, balanced accuracy (equal to macro recall) and log-loss. Uncertainty is estimated with 2,000 bootstrap resamples of the test images [22]; pairs of models are compared on the same images with a paired bootstrap of the accuracy difference and an exact McNemar test [23], which tests whether, on the images where exactly one of the two models is right, each model is right equally often.

4 Experimental setup and hyperparameters

4.1 Preprocessing

Every image is decoded to 8-bit RGB, and each model uses the input size and normalisation of its weights. For **evaluation** (validation, test, and the feature extraction of model C) the short side is resized to the input size divided by the crop fraction (0.875 for A and B, 0.9 for C) and the centre square is cut out: 183→160 px bilinear for A, 256→224 px bicubic for B, 427→384 px bicubic for C. Pixels are scaled to $[0, 1]$ and normalised per channel (mean and standard deviation 0.5 for A and C; ImageNet statistics for B).

4.2 Augmentation (training data only)

A and B are trained with `RandomResizedCrop` (aspect ratio 0.75–1.33), horizontal flips ($p = 0.5$) and RandAugment with two operations of magnitude 9 [24]; A also uses colour jitter 0.2 and a wider crop range (scale 0.35–1 instead of 0.5–1), because a network trained from scratch needs stronger regularisation (Figure 3). Validation and test images always receive the deterministic evaluation transform. The transformations are realistic for this domain: web photos of parts are taken at arbitrary zoom and framing (random crops), components have no canonical left–right orientation (horizontal flips), and lighting, white balance and compression vary between shops and phones (RandAugment's photometric operations, colour jitter). Vertical flips were not used because printed text and lighting in product shots have a natural "up", and MixUp/CutMix were not used because blended components cannot occur. Model C uses no augmentation: its features are computed once from the evaluation crop, which keeps the frozen-feature protocol simple and cheap.

4.3 Hyperparameters

Table 3 lists every training setting. Settings differ where the architectures require it. A randomly initialised CNN needs a high learning rate, many epochs and strong augmentation. A pretrained network needs a small learning rate and few epochs. A linear head on frozen, standardised features needs neither augmentation nor a learning-rate schedule. The recipe of B was fixed earlier in the project's grouped 5-fold cross-validation benchmark on the 2,512 development images only (`results/benchmark.csv`), and so was A's apart from its schedule: the benchmark's 120-epoch budget stopped A before it had converged, and the improvement study of Section 6.3, again on the development folds only, lengthened it to 200 epochs (patience 40). The 418 test images were never used for any of this. The only choice made here is the weight decay of model C's head, selected on the validation set from $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ (Table B.4, Appendix B). The 3 smallest values (10^{-4} to 10^{-2}) reached the identical best validation macro-F1, because AdamW's

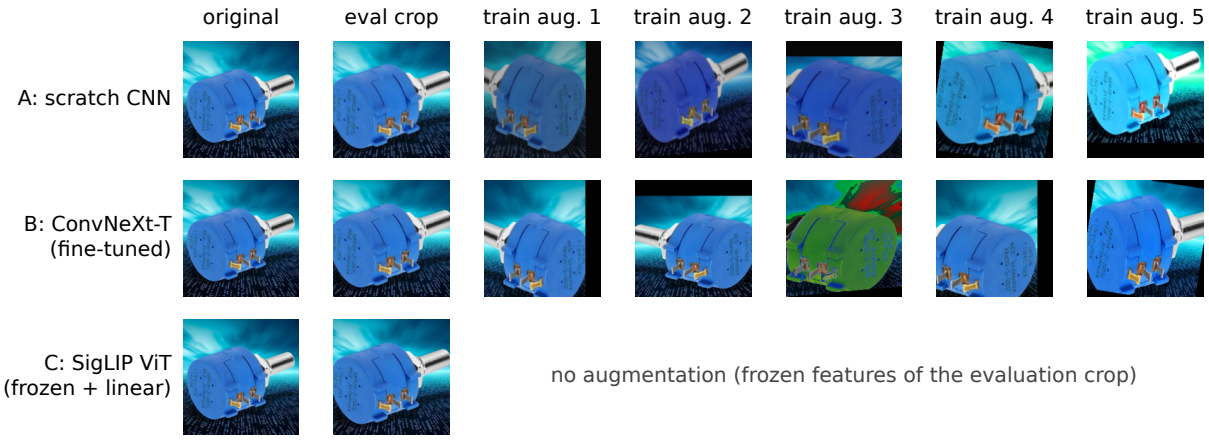


Figure 3: One training image, its evaluation crop and five random training augmentations per model. Model C uses frozen features of the evaluation crop and therefore no augmentation.

decoupled decay [25] is negligible over a few dozen epochs at this learning rate; the tie was broken by the rule, fixed before the grid was run, that the smallest value wins. A and B use label smoothing 0.1 [26] and a cosine schedule [27]. All models are early-stopped on validation macro-F1, and the best validation epoch is tested.

Table 3: Training settings (`results/itri626/hyperparameters.csv`, `results/itri626/runs/*.json`). Epochs are counted from 1.

	A: scratch CNN	**B**: ConvNeXt-T (fine-tuned)	**C**: SigLIP ViT (frozen + linear)
MODEL			
Model / variant	VGG-style CNN (scratch), width 32, 4 stages, dropout 0.3	ConvNeXt-Tiny, stochastic depth (drop-path rate) 0.1	ViT-So400m/14 (SigLIP), frozen; `nn.Linear(1152, 20)` head
Pretrained weights	none (He initialisation)	`convnext_tiny.fb_in22k_ft_in1k` (ImageNet-22k→1k)	`vit_so400m_patch14_siglip_384.webli` (WebLI)
Trainable / total params	1,187,636 / 1,187,636	27,835,508 / 27,835,508	23,060 / 428,248,660
DATA AND AUGMENTATION			
Input size (px)	160×160	224×224	384×384
Augmentation (training only)	RandomResizedCrop (scale 0.35–1), h-flip 0.5, RandAugment (2, 9), colour jitter 0.2	RandomResizedCrop (scale 0.5–1), h-flip 0.5, RandAugment (2, 9)	none (features of the evaluation crop)
Class weights / sampling	none (uniform shuffling)	none (uniform shuffling)	none (uniform shuffling)
Batch size; grad. accumulation	64; none	32; none	256; none
OPTIMISATION			
Optimiser	AdamW, grad. clip 1.0	AdamW, grad. clip 1.0	AdamW
Learning rate	0.002	0.0002	0.001
LR schedule	linear warm-up 5 epochs, cosine to 0	head only in epoch 1, then all layers; warm-up 2 epochs, cosine to 0; layer-wise LR decay 0.8	constant
Weight decay	0.05 (fixed)	0.05 (fixed)	0.0001 (grid on val., Table B.4)
Loss	cross-entropy, label smoothing 0.1	cross-entropy, label smoothing 0.1	cross-entropy, label smoothing 0.0
REGULARISATION AND STOPPING			
Early stopping	val. macro-F1, patience 40	val. macro-F1, patience 6	val. macro-F1, patience 20
Epochs (max / run / best)	200 / 200 / 184	20 / 20 / 19	200 / 34 / 14
RUNTIME			
Numerical precision	mixed fp16	mixed fp16	fp16 features, fp32 head
Random seed (training; split)	0; 42	0; 42	0; 42
Training time	390 s (GPU)	120 s (GPU)	1.7 s for all 4 grid runs (CPU, cached features)

4.4 Software, hardware and reproducibility

The models are implemented in PyTorch [28] with `timm` [10]; scikit-learn [7] provides the split and the reference probe, and uv [29] pins the environment. B was trained on one NVIDIA GeForce RTX 4060 Ti with 7.7 GiB of memory (all versions and the hardware are in Table B.1, Appendix B). The final run of A was trained after the improvement study (Section 6.3) on a second machine, one NVIDIA GeForce RTX 4070 Laptop GPU with 8.0 GiB (Windows 11; the same code, lock file and seed; its software versions are recorded in `results/itri626/runs/A\_scratch\_cnn.json`). Peak GPU memory was 435 MB for A and 2,312 MB for B, and head C was trained on the CPU from cached features. The training seed is 0 and the split seed 42. GPU training of A is not bit-for-bit reproducible (non-deterministic cuDNN kernels), so a re-run can differ slightly. All result files, the executed notebook `notebooks/ITRI626_four_models.ipynb` and the commands in Appendix D are in the repository.

Fairness. All three models use the same 2,010 training, 502 validation and 418 test images, the same selection rule (best validation macro-F1) and a single test evaluation each, with no test-time augmentation. Tuning effort is comparable: the recipes of A and B come from the earlier development-set benchmark, and C's only tuned setting is a four-value weight-decay grid on the validation set (Table B.4). Budgets differ only where the architecture requires it (Table 3): A receives 200 epochs and converges within them (best epoch 184), whereas B begins to overfit within 20. B and C also differ in pretraining data, which we treat as part of each method (Section 6.5). No test number led to a change of any setting.

5 Results and model comparison

5.1 Overall test performance

Table 4 gives the test metrics. The ranking is the same for every metric: model C (accuracy 0.823, macro-F1 0.817) > model B (0.636, 0.628) > model A (0.400, 0.392), and the 95% bootstrap intervals do not overlap. The paired comparisons (Table 5) are decisive: B is right where A is wrong on 119 images against 20 the other way ($p = 2.4 \times 10^{-18}$), and C beats B on 92 images against 14 ($p = 3.1 \times 10^{-15}$). Because the classes are nearly balanced, balanced accuracy and macro and weighted averages agree. Test accuracy is slightly above validation accuracy at the selected epoch for every model (A 0.378, B 0.618, C 0.803), within the sampling variation of two sets of about 450 images. C also has the lowest log-loss (0.614), i.e. its probabilities are the most reliable, which the reject option in Section 6.2 exploits. The reference rows confirm C: a scikit-learn logistic-regression probe on the same features (R1) scores 0.833, 4 images more than C, and the project's deployed model (R2: probe fitted on training plus validation data, fused with the zero-shot text head, i.e. SigLIP's text encoder scoring the 20 class names) scores 0.837. R2 is not a fourth model: it is C refitted on more data and is reported only to show what the recommended approach reaches in deployment.

5.2 Learning curves

Figure 4 shows training and validation loss and validation accuracy and macro-F1 per epoch; Table B.2 (Appendix B) gives the values at the selected epoch. Losses are plain cross-entropy.

- **A** improves slowly for most of its 200-epoch budget and is selected at epoch 184. There its training accuracy is 0.83 but its validation accuracy only 0.378. The validation loss is lowest at epoch 81 (2.447; chance level $\ln 20 = 3.00$) and then rises to 2.519 while validation accuracy and macro-F1 still improve: the network grows more confident without changing many decisions, and in its second hundred epochs it also fits the training set (training cross-entropy 0.712). The 120-epoch schedule of the earlier benchmark stopped it before this point (Section 6.3).

Table 4: Test-set metrics ($n = 418$, same images for all models; `results/itri626/metrics.json`). Bold: best main model. CI: 95% percentile bootstrap (2,000 resamples).

Model	Acc.	Bal. acc.	Macro P	Macro R	Macro F1	Weighted F1	Log loss
A: scratch CNN	0.400	0.391	0.399	0.391	0.392	0.400	2.361
95% CI	[0.354, 0.445]				[0.342, 0.432]		
B: ConvNeXt-T (fine-tuned)	0.636	0.630	0.642	0.630	0.628	0.632	1.346
95% CI	[0.591, 0.682]				[0.578, 0.668]		
C: SigLIP ViT (frozen + linear)	**0.823**	**0.821**	**0.827**	**0.821**	**0.817**	**0.820**	**0.614**
95% CI	[0.787, 0.859]				[0.774, 0.851]		
Reference rows (same frozen SigLIP features as C; not additional models)							
R1: SigLIP + sklearn probe	0.833	0.830	0.838	0.830	0.829	0.832	0.606
R2: deployed fused model	0.837	0.836	0.843	0.836	0.834	0.836	0.527

Table 5: Paired comparisons on the same 418 test images: accuracy difference Δ as a proportion (×100 = percentage points, as in the text), paired-bootstrap interval, discordant counts, exact McNemar test.

Pair	Δ accuracy	95% CI	Only 1st right	Only 2nd right	McNemar p
B − A	+0.237	+0.189 to +0.289	119	20	2.4×10^{-18}
C − B	+0.187	+0.141 to +0.232	92	14	3.1×10^{-15}
C − A	+0.423	+0.373 to +0.474	184	7	1.1×10^{-45}

- **B** profits from its pretrained features at once: validation accuracy passes 0.5 in epoch 4 and plateaus near 0.618, while training accuracy rises to 0.91 and validation loss stays at about 1.431, i.e. a 28-million-parameter network overfits a small, noisy training set. The cosine schedule and early stopping (best epoch 19 of 20) keep it from getting worse.
- **C** passes 0.75 validation accuracy in epoch 3 and reaches 0.803. Training accuracy goes to 1.00, since 1,152 standardised features are almost linearly separable for 2,010 images. Validation loss is lowest in epoch 10 (0.756) and then creeps up while accuracy and macro-F1 stay flat: the head grows more confident without changing many decisions. Early stopping selected epoch 14.

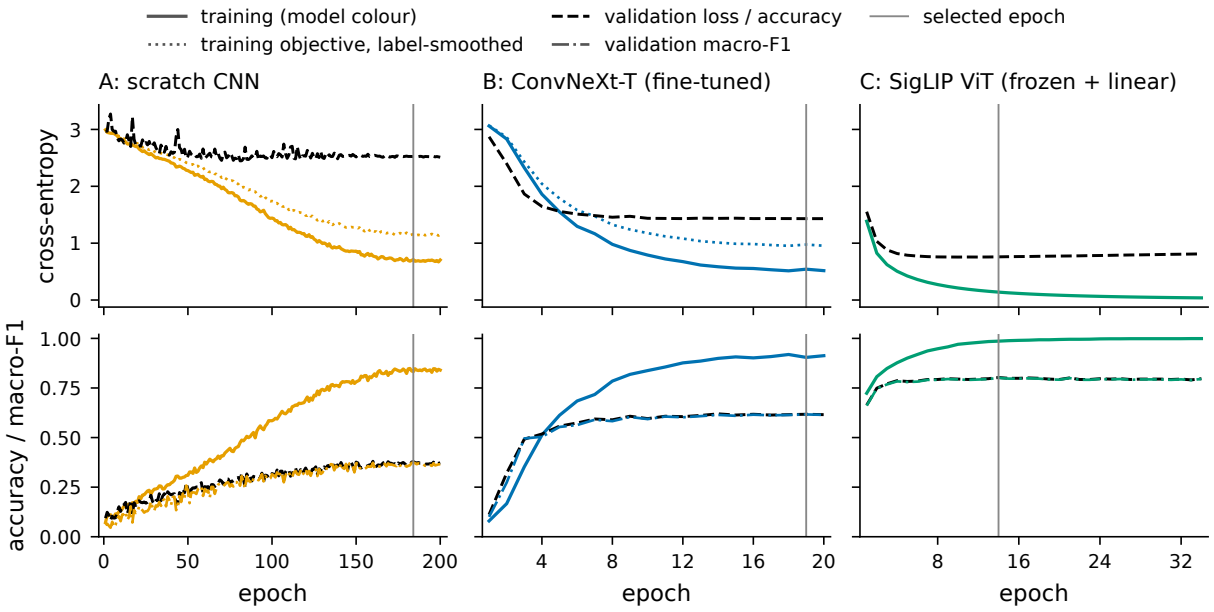


Figure 4: Learning curves (one column per model). Top: cross-entropy; dotted: label-smoothed training objective (A, B). Bottom: training accuracy, validation accuracy and macro-F1. Training curves of A and B are measured on augmented batches; the grey line marks the epoch whose weights were tested.

5.3 Per-class results and confusion matrices

Figure 5 plots the per-class test F1; the full per-class precision, recall and F1 values are in Table A.1 (Appendix A). Model C is best or tied for best in nearly every class; its weakest class is *resistor* (F1 0.61, recall 0.45) and its strongest *switch* (0.96). B is weakest on *fuse* (F1 0.34, recall 0.25). A exceeds F1 0.5 only for a few visually distinctive classes (*microcontroller board*, *switch*, *jumper wire*, *heat sink*). The confusion matrices (Figure 6) show A's errors spread over the whole matrix, B's concentrated in a few blocks, and C's matrix almost diagonal; Section 6.2 analyses the largest off-diagonal entries.

5.4 Correct and incorrect predictions

Figure 7 shows, for each model, six random correct predictions and its six most confident errors with true class, prediction and softmax confidence. Many of C's confident errors are images that do not show the labelled component: a DIP chip labelled *transistor*, a breadboard power module labelled *breadboard*, a fibre-optic pigtail and a coaxial plug labelled *jumper wire*, and an icon sheet and a "Capacitor vs Resistor" banner labelled *resistor*. A's most confident errors include a sheet of package drawings labelled *diode*, an LED arrow sign labelled *power supply module* and handwritten notes labelled *inductor coil*.

5.5 Agreement between the models and computational cost

All three models are right on 143 test images and all three wrong on 57 (Figure 8). 75 images are recognised only by C and 109 by B and C but not A, whereas C is wrong on only 17 images that A or B get right (examples in Figure 9).

Accuracy has a price (Table B.3 in Appendix B): C needs 31.1 ms per image on the GPU at batch size 1, about 11 times as long as B (2.79 ms), and 893.5 ms on an 8-thread CPU, about 49 times as long as B (18.4 ms); its frozen backbone weighs 1,633.6 MB in fp32.

6 Discussion, limitations and error analysis

6.1 Why the models differ

The ranking follows *how much relevant visual knowledge each model brings* to a 2,010-image problem, rather than depth or parameter count. **A** starts from random weights and must learn edges, textures, shapes and classes at once from about 100 images per class; its learning curves show it is data-limited, as does the improvement study of Section 6.3 (more epochs helped; more capacity, residual blocks, stronger augmentation and better initialisation did not), and its per-class F1 correlates with the number of training images (Spearman ρ over the 20 classes $+0.51$, against $+0.56$ for B and $+0.43$ for C). **B** re-uses ImageNet-22k features that already encode objects, materials and shapes, so it learns fast; but ImageNet contains few electronic parts, so the features are generic, and fine-tuning all 28 million parameters on 2,010 noisy images overfits (training accuracy 0.91 against 0.618 on validation). **C** uses a 428-million-parameter encoder pretrained on web image–text pairs. Its embedding already separates the classes: SigLIP zero-shot classification, with no training at all, reaches macro-F1 0.776 in the benchmark (Section 6.4). It is also the least affected by off-topic images (0.759 on flagged test images against 0.500 for B, Section 6.2), plausibly because such images resemble its web pretraining data. Freezing the backbone means label noise cannot distort it, and training the 23,060 head parameters takes 1.7 s for the whole grid on cached features.

6.2 Error analysis

Look-alike components. Table 6 lists the most frequent confusions. C's largest are *resistor→inductor coil* (4) and *jumper wire→connector* (3); B's are *resistor→crystal oscillator* (5) and *ic chip→microcontroller board* (4). These pairs share shapes: colour-banded axial bodies (some inductors look like fat resistors), cables with plugs, chips mounted on boards. *Resistor*

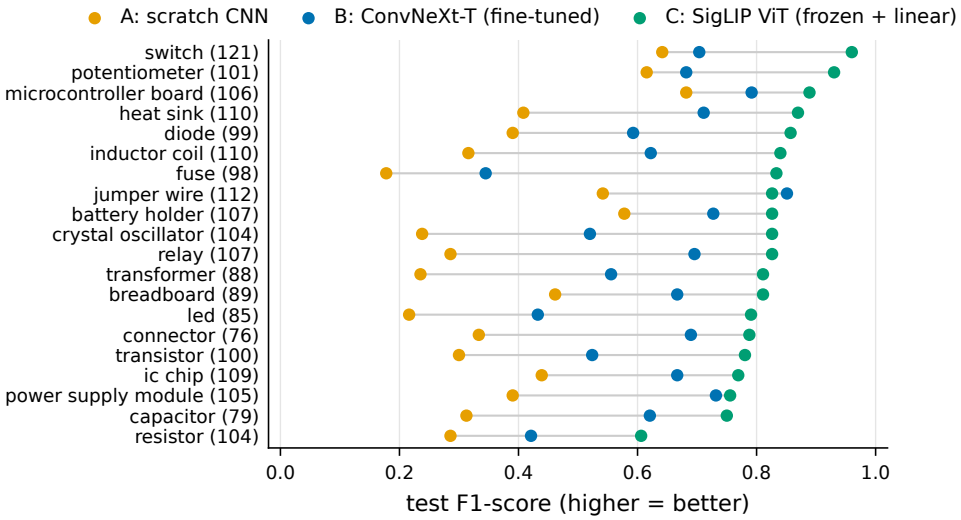


Figure 5: Per-class test F1 of the three models, sorted by model C; the number after each class is its training-set size.

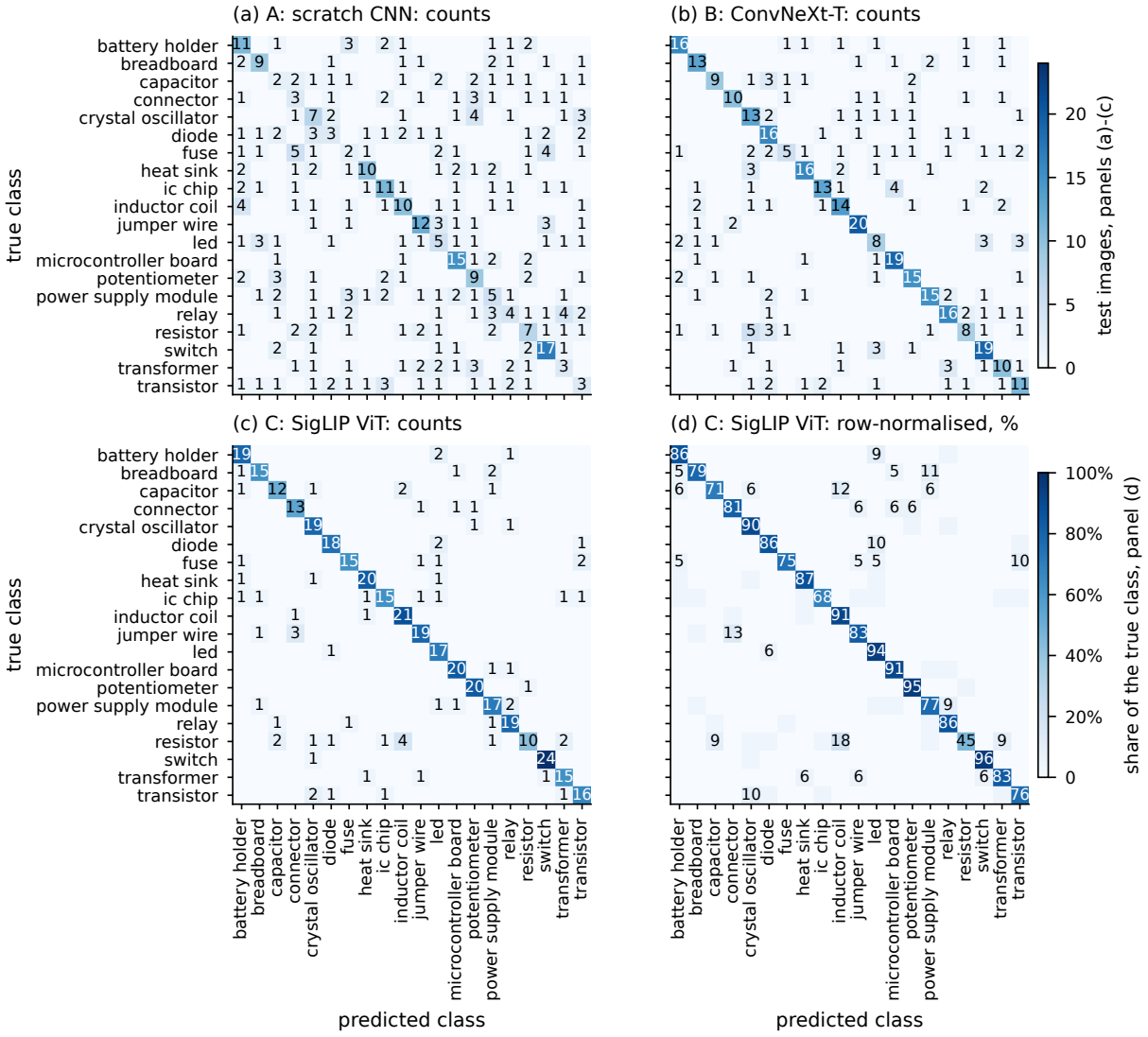
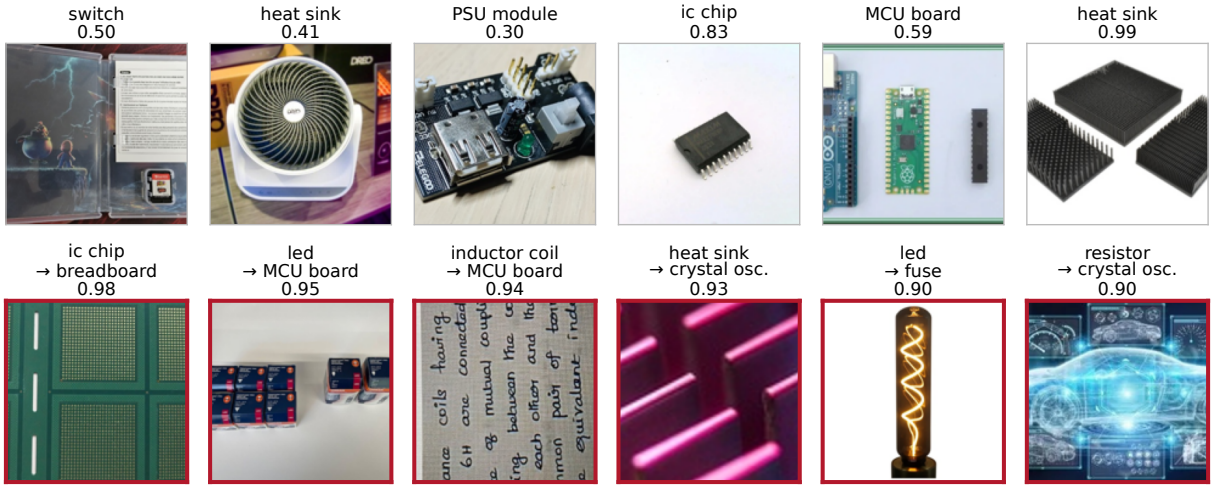
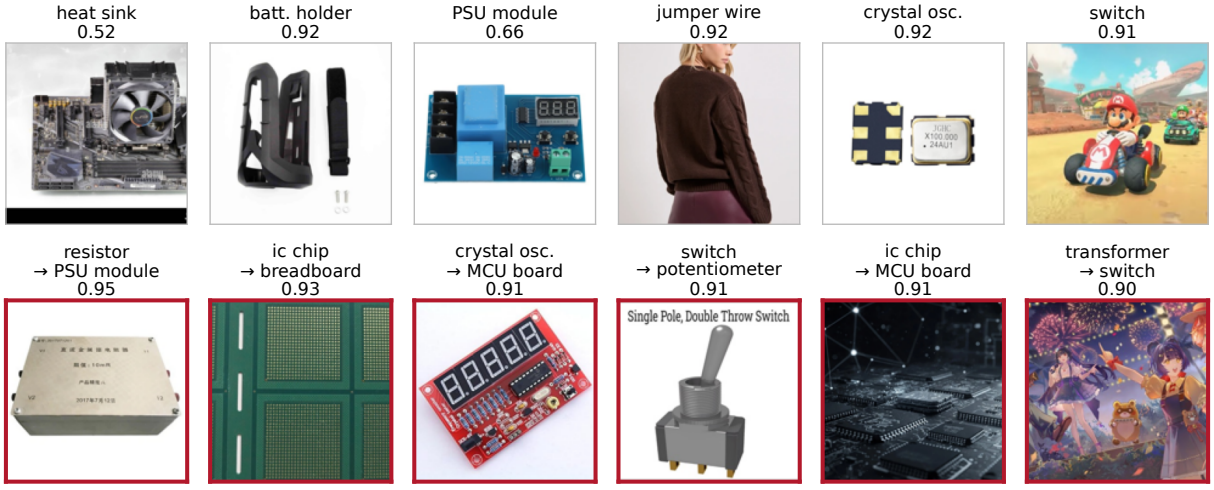


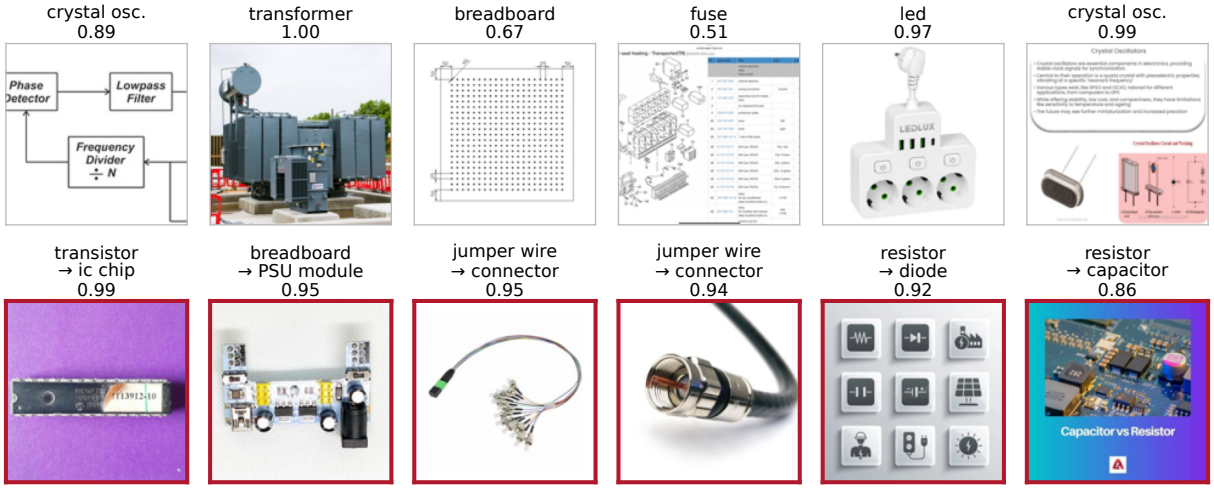
Figure 6: Test confusion matrices (rows: true class; columns: predicted class; $n = 418$). (a)–(c) Counts on one shared colour scale, so equal shades mean equal counts; empty cells are zero. (d) Model C row-normalised (the diagonal is the per-class recall); those of A and B are in the notebook.



(a) Model A (scratch CNN)



(b) Model B (ConvNeXt-T, fine-tuned)



(c) Model C (SigLIP ViT, frozen + linear)

Figure 7: Six random correct predictions (top row of each panel) and the six most confident errors (bottom row, red frame, *true* → *predicted*) of every model on the test set. Confidence is the softmax probability of the predicted class. Abbreviations: MCU board = microcontroller board, PSU module = power supply module, crystal osc. = crystal oscillator, batt. holder = battery holder.

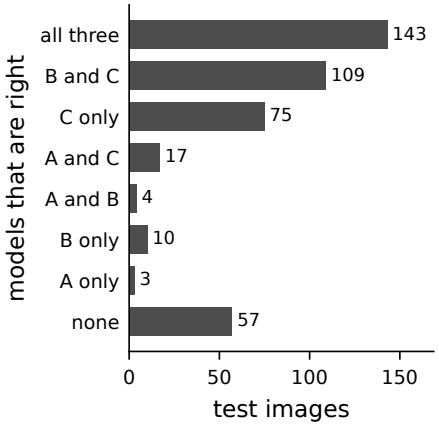


Figure 8: Test images by the set of models that classify them correctly.

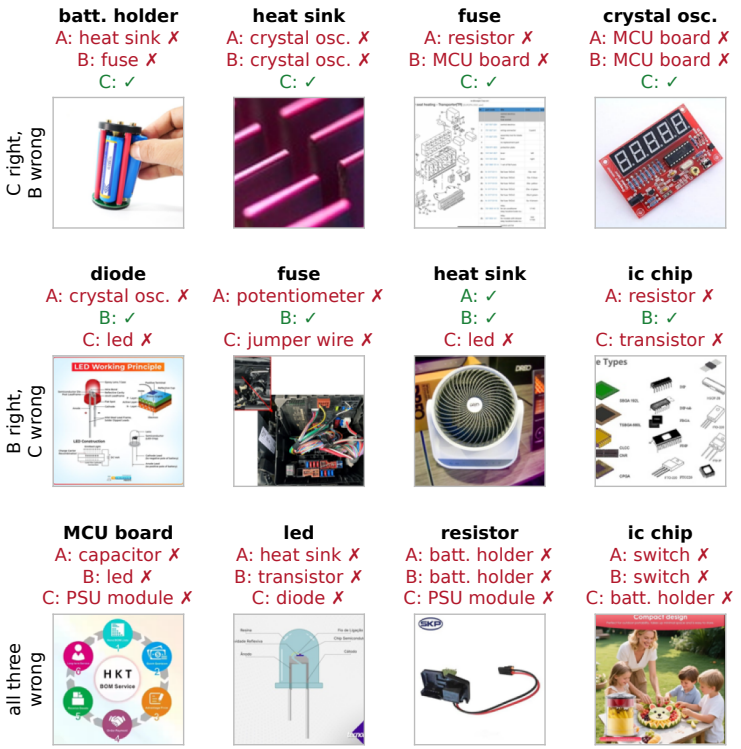


Figure 9: Disagreements. Rows: C right and B wrong; B right and C wrong; all three wrong. Each title gives the true class (bold) and each model's prediction; a green tick marks a correct and a red cross a wrong prediction. Abbreviations as in Figure 7.

is C's hardest class: precision 0.91 but recall only 0.45, partly because several test "resistors" are icon sheets, banners or schematics. A's confusions (e.g. *fuse→connector*, *fuse→switch*) are spread widely and often have no obvious visual explanation, consistent with a model that has not learned robust shape features. *Diode* and *led* are related devices that are often shown together in infographics (2 of C's errors are *diode→led*). Summed over the three models, the most frequent confusions are resistor → crystal oscillator (8); jumper wire → connector (7); transistor → diode (7); crystal oscillator → relay (6).

Data quality matters more than class imbalance. On the 306 test images that the CLIP screen did not flag, accuracy is 0.435 (A), 0.686 (B) and 0.846 (C); on the 112 flagged ones it is 0.304, 0.500 and 0.759. B loses most (19 points; A 13, C 9), and C stays far ahead on flagged images, plausibly because its pretraining has seen such web content. All three models fail on 57 test images, many of them infographics, diagrams or scenes without the labelled component, where the label rather than the prediction is often questionable (Figure 9). Class imbalance, by contrast,

Table 6: Most frequent test confusions (true → predicted) per model, from the confusion matrices in `results/itri626/metrics.json`; equal counts are listed alphabetically. MCU board: microcontroller board.

	A: scratch CNN		B: ConvNeXt-T (fine-tuned)		C: SigLIP ViT (frozen + linear)	
#	True → predicted	*n*	True → predicted	*n*	True → predicted	*n*
1	crystal oscillator → relay	5	resistor → crystal oscillator	5	resistor → inductor coil	4
2	inductor coil → resistor	4	ic chip → MCU board	4	jumper wire → connector	3
3	transistor → diode	4	capacitor → diode	3	battery holder → led	2
4	battery holder → heat sink	3	heat sink → crystal oscillator	3	breadboard → power supply module	2
5	capacitor → relay	3	led → switch	3	capacitor → inductor coil	2
6	connector → potentiometer	3	led → transistor	3	diode → led	2

is not an important factor: the size ratio is only 1.61, and C's per-class F1 is not significantly related to class size ($\rho = +0.43$, $p = 0.06$).

Confidence is informative. C's mean softmax confidence is 0.87 on correct and 0.47 on wrong predictions (B: 0.76 and 0.43), so a reject option can catch many errors [30, 31]. The deployed model uses one: at a confidence threshold of 0.5 it answers 87.1% of the test images and is right on 0.909 of those.

6.3 Can the scratch CNN be improved?

Model A's score invites the question whether a better recipe, rather than pretraining, would close the gap. A 19-experiment study answered it on the development folds only (`docs/SCRATCH\_CNN\_IMPROVEMENT\_REPORT.md`; log in `docs/SCRATCH\_CNN\_IMPROVEMENT\_LOG.md`; each change screened on folds 0–2 and confirmed on all five, with single-run noise of ±2–3 points). Input resolution (224 and 256 px), MixUp, width (2.2× and 4× the parameters), depth and residual blocks, label smoothing 0.2, flip test-time augmentation, whitening and identity initialisation, stronger augmentation, dropout 0.5, weight averaging and whole-picture resizing all changed nothing or made it worse. One thing helped (Table 7): training to convergence (200 instead of 120 epochs) added 3.5 points of five-fold macro-F1 on every fold and 5.6 points on the test split; that schedule is the model A reported in this paper. It needs ten times the epochs of the fine-tuned network (200 against 20) and still overfits: 0.83 training accuracy against 0.378 on validation at the selected epoch. On the 407 out-of-source Wikimedia photographs the scratch network falls to 0.257 macro-F1 against 0.816 for the deployed SigLIP model. The lesson is the main comparison's: features learned from 2,010 photographs of one web scrape describe that scrape, and the limit is the data, not the training budget.

6.4 Supporting evidence from the wider study

The project's broader study (details in Appendix C) supports the comparison. Grouped 5-fold cross-validation over eleven families reproduces the ordering (macro-F1: scratch CNN 0.355 with the benchmark's 120-epoch schedule, 0.390 with the 200-epoch schedule of Section 6.3; ConvNeXt-T fine-tuned 0.641, frozen SigLIP So400m probe 0.824), and every ImageNet-pretrained fine-tune (0.547–0.654) stays below SigLIP zero-shot classification without any training (0.776). The dataset's original file-level split leaks duplicates and inflates linear-probe accuracy by 5.5 to 13.6 points, more for weaker models. On 407 fresh Wikimedia Commons photographs the deployed SigLIP-based model (R2) scored 0.821 accuracy, close to its in-distribution result.

6.5 Limitations

1. **One split, one training seed.** With about 21 test images per class, one image moves a per-class F1 by up to about 0.05, and accuracy differences below about 4 points would not be significant. The cross-validation benchmark gives the same ordering.
2. **Noisy labels and scope.** Labels are unreviewed search terms and 27.8% of the images are flagged as possibly off-topic (by an automated screen that no person verified). Measured accuracy therefore underestimates performance on clean photographs. Because no person checked the test labels either, per-class scores for homonym classes (*switch*, *transformer*) partly measure the scrape rather than the model.

Table 7: Scratch-CNN improvement study. Development: grouped 5-fold macro-F1 (mean ± SD over folds). Test and holdout: one training run on the fixed split, scored once (`results/scratch\_cnn\_improvement/`). The 120-epoch row is the model A of the previous build of this report; R2 is the deployed model of Table 4.

Recipe	Epochs	Dev. 5-fold macro-F1	Test macro-F1	Test acc.	Holdout macro-F1	Holdout acc.
A, previous build	120	0.355 ± 0.015	0.336	0.354	–	–
A, this report	200	0.390 ± 0.007	0.392	0.400	0.257	0.283
Deployed SigLIP model (R2)	–	–	0.834	0.837	0.816	0.821

3. **Screen and model share a lineage.** The off-topic flags come from a CLIP model, a relative of C's SigLIP encoder, so the flagged/unflagged comparison in Section 6.2 may favour C.
4. **Residual near-duplicates.** The near-duplicate rule (pHash and CLIP thresholds) can miss heavily edited copies, so some train–test similarity may remain.
5. **Unequal pretraining and budgets.** C's backbone was trained on far more data than B's, so the comparison measures model and pretraining together. The budgets of A and B were fixed by earlier cross-validation, not tuned exhaustively. A was then trained to convergence, and the study of Section 6.3 found no further recipe gain, so the gap of 42.3 points to C is not a budget artefact.
6. **Cost.** C has 428.2 million parameters and needs about 893.5 ms per image on a CPU, which rules it out for microcontrollers or real-time video without a GPU.

7 Conclusion

The answer to our question is clear: a frozen web-pretrained encoder with a linear head beats both fine-tuning and training from scratch on this data. On one leakage-free split and the same 418 test images the three model families rank: a frozen SigLIP So400m ViT with a linear head (accuracy 0.823, macro-F1 0.817), a fine-tuned ConvNeXt-T (0.636, 0.628) and a VGG-style CNN trained from scratch (0.400, 0.392). The differences are statistically clear and confirmed by the 5-fold benchmark. Transfer learning is the decisive factor: ImageNet fine-tuning adds 23.7 percentage points over training from scratch, and large-scale image–text pretraining another 18.7. Trying to improve the scratch network directly (Section 6.3) did not change this picture: training it to convergence gained 3.5 points, and nothing in architecture, regularisation or initialisation gained anything.

Recommended model: C, the frozen SigLIP So400m ViT with a linear softmax head. It is the most accurate by a wide margin, the most robust to off-topic web images, confident enough on correct answers for a reject option, and the cheapest to train (23,060 parameters, learned in seconds), provided a GPU or about a second per CPU image is acceptable. Where inference cost dominates (mobile devices, real-time video), the fine-tuned ConvNeXt-T is the practical second choice: about 11 times faster on a GPU, at about 18.7 percentage points lower. R2, the deployed variant of model C (probe refitted on training plus validation data and fused with the text head), scores 0.837 and is what we would ship. The clearest route to further gains is better data rather than a bigger model: relabelling the audited issues and cleaning the classes contaminated by homonyms (*switch*, *transformer*).

Lessons learned. On a few thousand noisy web images the choice of pretrained representation matters far more than the classifier trained on top, and a grouped, duplicate-aware split is needed before any of these differences can be trusted.

References

[1] Y. Shao, L. Zhou, F. Tang, X. Shi, D. Chen, and S. Xia, "Comparative performance of finetuned ImageNet pre-trained models for electronic component classification," in *Proc. 7th Int. Conf. Universal Village (UV)*, pp. 1–10, 2024.

[2] L. Zhou and L. Zhang, "A novel convolutional neural network for electronic component classification with diverse backgrounds," in *Modeling and Simulation in Manufacturing*, pp. 191–207, World Scientific, 2025.

[3] Q. Ling and N. A. M. Isa, "Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey," *IEEE Access*, vol. 11, pp. 15921–15944, 2023.

[4] mdfaisalahmed025, "Electronic Component Recognizer." Kaggle dataset (20 classes), https://www.kaggle.com/datasets/mdfaisalahmed025/electronic-component-recognizer. Accessed 17 October 2026.

[5] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language

supervision," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139 of *Proc. Mach. Learn. Res.*, pp. 8748–8763, 2021.

[6] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, 2021.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[8] B. Barz and J. Denzler, "Do we train on test data? Purging CIFAR of near-duplicates," *J. Imaging*, vol. 6, no. 6, p. 41, 2020.

[9] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[10] R. Wightman, "PyTorch Image Models (timm)." GitHub repository, https://github.com/huggingface/pytorch-image-models, 2019.

[11] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[12] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn. (ICML)*, vol. 37 of *Proc. Mach. Learn. Res.*, pp. 448–456, 2015.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, pp. 1026–1034, 2015.

[14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 11966–11976, 2022.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770–778, 2016.

[16] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[17] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[18] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, pp. 3320–3328, 2014.

[19] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 2656–2666, 2019.

[20] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[21] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, pp. 11941–11952, 2023.

[22] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[23] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[24] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, "RandAugment: Practical automated data augmentation with a reduced search space," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, pp. 3008–3017, 2020.

[25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[26] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 2818–2826, 2016.

[27] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 32, 2019.

[29] Astral Software, "uv: an extremely fast Python package and project manager." https://docs.astral.sh/uv/, 2024.

[30] C. Chow, “On optimum recognition error and reject tradeoff,” *IEEE Trans. Inf. Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[31] Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

A Full per-class results

Table A.1 gives the per-class test precision, recall and F1 of the three main models, and Table A.2 those of the two reference rows on the same test images. The per-class image counts of every split are in Table 1.

Table A.1: Per-class test precision (P), recall (R) and F1 (n = test images per class; `results/itri626/per_class.csv`). Best F1 per class in bold.

		A: scratch CNN			B: ConvNeXt-T			C: SigLIP ViT		
Class	n	P	R	F1	P	R	F1	P	R	F1
battery holder	22	0.57	0.59	0.58	0.73	0.73	0.73	0.79	0.86	**0.83**
breadboard	19	0.45	0.47	0.46	0.65	0.68	0.67	0.83	0.79	**0.81**
capacitor	17	0.33	0.29	0.31	0.75	0.53	0.62	0.80	0.71	**0.75**
connector	16	0.36	0.31	0.33	0.77	0.62	0.69	0.76	0.81	**0.79**
crystal oscillator	21	0.24	0.24	0.24	0.45	0.62	0.52	0.76	0.90	**0.83**
diode	21	0.40	0.38	0.39	0.48	0.76	0.59	0.86	0.86	**0.86**
fuse	20	0.16	0.20	0.18	0.56	0.25	0.34	0.94	0.75	**0.83**
heat sink	23	0.38	0.43	0.41	0.73	0.70	0.71	0.87	0.87	**0.87**
ic chip	22	0.47	0.41	0.44	0.76	0.59	0.67	0.88	0.68	**0.77**
inductor coil	23	0.40	0.26	0.32	0.64	0.61	0.62	0.78	0.91	**0.84**
jumper wire	23	0.52	0.57	0.54	0.83	0.87	**0.85**	0.83	0.83	0.83
led	18	0.21	0.22	0.22	0.42	0.44	0.43	0.68	0.94	**0.79**
microcontroller board	22	0.68	0.68	0.68	0.73	0.86	0.79	0.87	0.91	**0.89**
potentiometer	21	0.67	0.57	0.62	0.65	0.71	0.68	0.91	0.95	**0.93**
power supply module	22	0.42	0.36	0.39	0.79	0.68	0.73	0.74	0.77	**0.76**
relay	22	0.24	0.36	0.29	0.67	0.73	0.70	0.79	0.86	**0.83**
resistor	22	0.30	0.27	0.29	0.50	0.36	0.42	0.91	0.45	**0.61**
switch	25	0.61	0.68	0.64	0.66	0.76	0.70	0.96	0.96	**0.96**
transformer	18	0.25	0.22	0.24	0.56	0.56	0.56	0.79	0.83	**0.81**
transistor	21	0.32	0.29	0.30	0.52	0.52	0.52	0.80	0.76	**0.78**
Macro average	418	0.40	0.39	0.39	0.64	0.63	0.63	0.83	0.82	0.82

Table A.2: Per-class test results of the reference rows (`results/itri626/metrics.json`, key `references`). R1: scikit-learn logistic-regression probe (inverse regularisation $C = 0.01$) on the same frozen SigLIP features and training split as C. R2: the deployed model (probe fitted on train + validation, fused with the zero-shot text head, temperature 0.898).

		R1: sklearn probe			R2: deployed		
Class	n	P	R	F1	P	R	F1
battery holder	22	0.77	0.91	0.83	0.83	0.86	0.84
breadboard	19	0.79	0.79	0.79	0.84	0.84	0.84
capacitor	17	0.80	0.71	0.75	0.81	0.76	0.79
connector	16	0.76	0.81	0.79	0.76	0.81	0.79
crystal oscillator	21	0.73	0.90	0.81	0.76	0.90	0.83
diode	21	0.86	0.86	0.86	0.86	0.86	0.86
fuse	20	0.94	0.85	0.89	0.94	0.80	0.86
heat sink	23	0.91	0.87	0.89	0.87	0.87	0.87
ic chip	22	0.94	0.73	0.82	0.94	0.77	0.85
inductor coil	23	0.84	0.91	0.88	0.91	0.87	0.89
jumper wire	23	0.83	0.83	0.83	0.76	0.83	0.79
led	18	0.73	0.89	0.80	0.68	0.94	0.79
microcontroller board	22	0.87	0.91	0.89	0.91	0.91	0.91
potentiometer	21	0.91	0.95	0.93	0.91	1.00	0.95
power supply module	22	0.71	0.77	0.74	0.70	0.73	0.71
relay	22	0.79	0.86	0.83	0.79	0.86	0.83
resistor	22	1.00	0.59	0.74	1.00	0.55	0.71
switch	25	0.96	0.92	0.94	0.96	0.96	0.96
transformer	18	0.78	0.78	0.78	0.78	0.78	0.78
transistor	21	0.84	0.76	0.80	0.85	0.81	0.83
Macro average	418	0.84	0.83	0.83	0.84	0.84	0.83

B Additional dataset and training details

Table B.1 lists the software and hardware of the training run.

Table B.2 gives the state of each model at its selected epoch, Table B.3 model size and inference time, Figure B.1 the class distribution before and after cleaning and per split (the numbers are in Table 1), and Table B.4 the weight-decay grid of model C's head.

Table B.1: Software and hardware of the training run of B and C (`results/itri626/environment.json`); the final run of A used a second machine (Section 4.4).

Component	Version / specification
Python	3.12.14
PyTorch / torchvision	2.11.0+cu128 / 0.26.0+cu128
timm (architectures and weights)	1.0.30
open_clip_torch	3.3.0
scikit-learn / NumPy / pandas	1.9.1 / 2.5.4 / 3.0.6
Pillow / matplotlib	13.0.0 / 3.11.2
ONNX / onnxruntime (deployment)	1.23.2 / 1.31.0
CUDA (torch build) / cuDNN	12.8 / 91900
GPU	NVIDIA GeForce RTX 4060 Ti, 7.7 GiB memory (compute capability 8.9)
NVIDIA driver	615.71.09
CPU / RAM	AMD Ryzen 7 8700F 8-Core Processor (16 logical cores) / 30.7 GiB
Operating system	Linux-7.2.7-1-cachyos-x86_64-with-glibc2.44
Environment manager	uv (lock file `uv.lock`)

Table B.2: State at the selected (best validation macro-F1) epoch (`results/itri626/histories.json`). CE: cross-entropy.

Model	Epochs run	Best epoch	Train acc.	Val. acc.	Val. macro-F1	Train CE	Val. CE
A: scratch CNN	200	184	0.832	0.378	0.374	0.712	2.519
B: ConvNeXt-T (fine-tuned)	20	19	0.904	0.618	0.617	0.542	1.431
C: SigLIP ViT (frozen + linear)	34	14	0.986	0.803	0.801	0.139	0.759

Table B.3: Model size and inference time (`results/itri626/efficiency.csv`): forward pass of a preprocessed tensor after warm-up, fp16 on the GPU, fp32 on 8 CPU threads, without JPEG decoding and resizing. C's checkpoint holds only the head; the backbone is the public weight file.

Model	Params (M)	Trainable params	Input (px)	Checkpoint (MB)	GPU, $b = 1$ (ms/img)	GPU, $b = 32$ (ms/img)	CPU, $b = 1$ (ms/img)
A: scratch CNN	1.19	1,187,636	160	4.56	0.52	0.046	1.7
B: ConvNeXt-T (fine-tuned)	27.84	27,835,508	224	106.24	2.79	0.803	18.4
C: SigLIP ViT (frozen + linear)	428.25	23,060	384	0.10	31.10	22.475	893.5

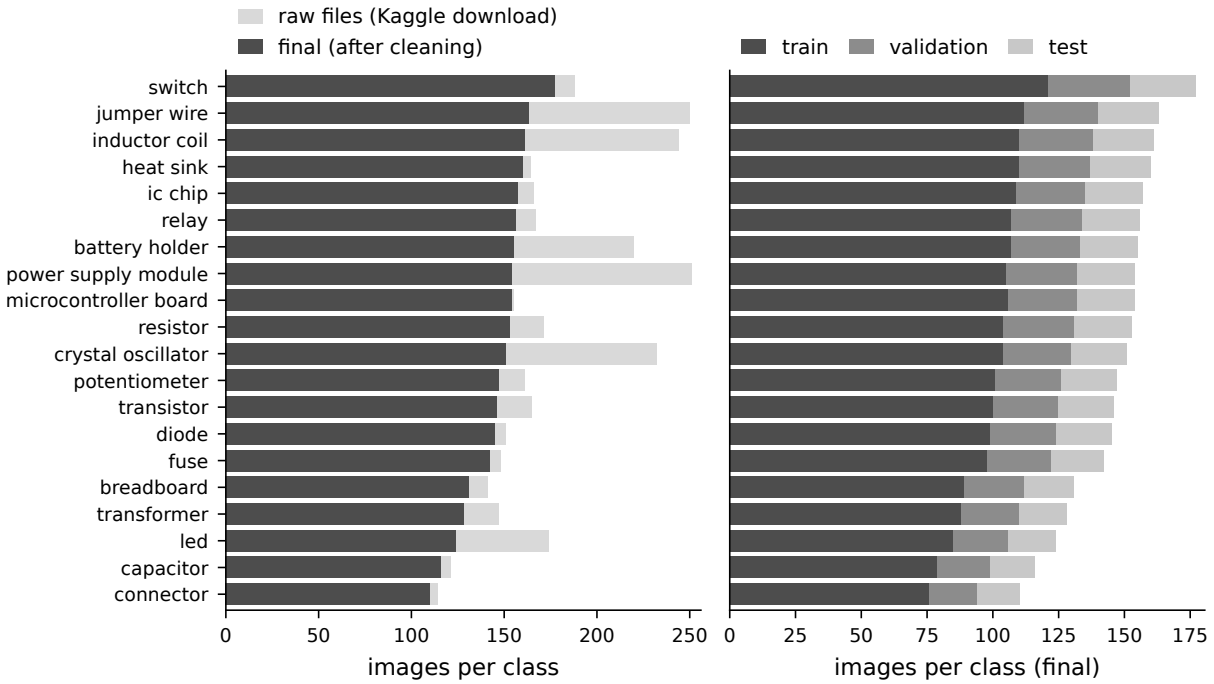


Figure B.1: Left: images per class in the raw download (light) and after cleaning (dark). Right: the split of the cleaned images into training, validation and test sets (68.6 / 17.1 / 14.3%).

Table B.4: Weight-decay selection for model C's head on the validation set (learning rate 10^{-3}, batch 256, at most 200 epochs, patience 20; `results/itri626/head_grid.csv`). The 3-way tie was broken by the pre-set smallest-value rule. CE: cross-entropy.

Weight decay	Best epoch	Epochs run	Val. acc.	Val. macro-F1	Val. CE	Selected
0.0001	14	34	0.8028	0.8011	0.7595	yes
0.001	14	34	0.8028	0.8011	0.7594	
0.01	14	34	0.8028	0.8011	0.7594	
0.1	21	41	0.8008	0.7999	0.7736	

C Supporting analysis

This appendix details the supporting results summarised in Section 6.4 (full write-up: `docs/report/REPORT.md`). In Table C.1, fine-tuning used one fixed recipe per family, the probe is a logistic regression on frozen features, and zero-shot uses the model's text encoder with class-name prompts and no training.

Eleven-family benchmark. Grouped 5-fold cross-validation on the 2,512 development images (Table C.1) reproduces the ordering: scratch CNN 0.355 macro-F1, ConvNeXt-T fine-tuned 0.641, frozen SigLIP So400m probe 0.824. All six ImageNet-pretrained fine-tunes lie between 0.547 and 0.654, SigLIP zero-shot classification without training (0.776) beats all of them, and partial fine-tuning of the SigLIP backbone (0.812, three folds) did not beat the frozen probe.

Leakage of the original split. Under the dataset's original file-level split, 198 of the 853 deduplicated development images that it places in validation or test (23.2%) also have a copy in its training part (our own test images are excluded; Section 2.5 counts raw file entries instead), which inflates three linear probes by 5.5 (SigLIP), 11.0 (ConvNeXt-T) and 13.6 (ResNet-50) points (`results/leakage_simulation.csv`). Weaker models gain most, so leakage also compresses the gaps between models.

Out-of-source test and deployment. The deployed model (R2) was tested once on 407 Wikimedia Commons photographs collected after it was frozen (`data/web_holdout_v1/`): 0.821 accuracy and 0.816 macro-F1, close to its in-distribution result. Its weakest classes there are *led* (recall 0.47; 7 errors to *diode*), *switch* (0.52; real switches predicted *ic chip* 6 times, a consequence of the console-dominated training class) and *transformer* (0.67; 5 errors to *inductor coil*). It is exported to ONNX for an OpenCV demo and a Rust desktop application (`docs/DEMO.md`, `apps/gpui-demo/`).

Table C.1: Supporting benchmark: grouped 5-fold CV macro-F1 (mean ± SD over folds) on the development set (`results/benchmark.csv`). †Three folds only. Model C itself corresponds to the SigLIP probe column.

Family (variant)	Fine-tune	Frozen linear probe	Zero-shot
Classical: HOG + colour histogram + SVM	0.206 ± 0.017 (classical pipeline)		
Scratch CNN (model A's network, 120-epoch recipe)	0.355 ± 0.015	–	–
ResNet-50	0.580 ± 0.012	0.515 ± 0.018	–
MobileNetV3-L	0.547 ± 0.016	–	–
EfficientNetV2-S	0.608 ± 0.024	–	–
ConvNeXt-T (as model B)	0.641 ± 0.034	0.593 ± 0.021	–
ViT-B/16 (AugReg)	0.654 ± 0.032	–	–
Swin-T	0.639 ± 0.034	–	–
DINOv2 (FT: ViT-B/14; probe: ViT-L/14)	0.668 ± 0.024	0.667 ± 0.019	–
CLIP (FT: ViT-B/16; others: ViT-L/14@336)	0.702 ± 0.017	0.721 ± 0.019	0.609 ± 0.029
SigLIP So400m/14@384 (backbone of C)	0.812 ± 0.023†	0.824 ± 0.012	0.776 ± 0.018

D Reproduction

All commands run from the repository root with uv (Python 3.12). The images are not tracked in Git. Download the Kaggle dataset (Section 2.1) and unpack it to `data/raw/electronic-component-recognizer/` to rebuild `data/v2/images/` (2,930 JPEG files) with `tools/build_dataset_v2.py`; the result must reproduce manifest fingerprint `431315e87d2612cb`.

```
uv sync                    # runtime environment (pyproject.toml + uv.lock)
uv sync --group dev        # + pytest, ipykernel, jupyterlab
uv run --no-config tools/build_dataset_v2.py   # only if data/v2/images/ is missing
uv run python tools/export_splits.py --check    # verify data/v2/splits/*.csv
uv run python scripts/itri626_experiments.py    # train A, B, C; score test (GPU)
uv run jupyter nbconvert --to notebook --execute --inplace \
    notebooks/ITRI626_four_models.ipynb        # RETRAIN = False by default
```

```
uv run --no-project --with numpy --with scipy \
    python docs/itri626_report/make_tables.py   # report tables and numbers
cd docs/itri626_report && latexmk -pdf main.tex # this report
```

Setting `RETRAIN = True` in the first code cell of the notebook re-runs the experiment script before loading its outputs. `scripts/itri626_experiments.py --steps head reference collect` re-runs only the CPU parts (head C from cached features, the reference rows and the result collection). The model-A run reported here was not produced by `train_A` but imported from the improvement study's fold-0 run with `tools/itri626\_import\_run.py`, so that the test split was scored only once for that recipe; re-running `train_A` trains a new network and spends a second look.

Provenance of further numbers in the text. Cleaning ablation (Section 2.3, item 4): `results/phase1/cleaning_ablation_siglip_so400m.csv`; label audit (item 5): `results/phase1/label_issues_v2.csv`; CLIP screen and its top distractors (Sections 2.3 and 2.6): `data/v2/review/ood_suspects.csv`; off-topic flags of the test images (Section 6.2): `data/v2/splits/test.csv`; reject option of the deployed model: `results/final/final_test_v2.json`; full write-up of the wider study (Section 6.4): `docs/report/REPORT.md`. Every number macro and its source file are listed in `docs/itri626_report/generated/key_numbers.json`.

E Rubric compliance checklist

Table E.1 maps every requirement of the project brief to the place where it is addressed. The same list, with file paths, is in `docs/itri626_report/RUBRIC_CHECKLIST.md`.

Table E.1: Brief requirement → location in this report or repository.

Requirement (brief section)	Where it is addressed
Problem definition and motivation (2 marks)	
Classification task, aim, scope, motivation (brief §2, §4)	Section 1 (incl. research question and main findings)
Target classes and reasons for them (brief §2, §5)	Section 1, Table 1
Dataset selection and preparation (4 marks)	
Source, access link, licence, usage conditions (brief §5)	Section 2.1; README "ITRI626 submission"
Class definitions (brief §5)	Table 1
Images per class before and after cleaning (brief §5)	Table 1, Table 2, Figure B.1
Removed duplicates, corrupt files, irrelevant samples, uncertain labels (brief §5)	Section 2.3, Table 2
How labels were obtained, checked and corrected (brief §5)	Section 2.3 (last paragraph)
Class imbalance and how it was handled (brief §5)	Section 2.4; Section 6.2
Representative examples of every class (brief §5)	Figure 1
Ethics, privacy, lawful use, acknowledgement of internet data (brief §5)	Section 2.6, Section 2.1
Train/validation/test split; same test set for all models (brief §7.1)	Section 2.5; `data/v2/splits/`
Leakage prevention by group identity (brief §7.1)	Section 2.5, Section 2.3 item 3, Section 6.4
Random seed, images per split, class distribution per split (brief §7.1)	Section 2.5, Table 1
Model selection and methodology (4 marks)	
At least three models from three families (brief §6)	Section 3: plain CNN, ConvNeXt, ViT
At least one model with transfer learning (brief §6)	B (fine-tuning) and C (feature extraction), Section 3
Methods, transfer-learning approach, complete workflow (brief §6)	Section 3, Figure 2

Table E.1: (continued)

Requirement (brief section)	Where it is addressed
Same split for all models; fair, justified hyperparameters (brief §6)	Section 4, Table 3, Table B.4
Experimental setup and reproducibility (3 marks)	
Preprocessing steps (brief §7.2)	Section 4 (Preprocessing)
Augmentation, training data only, realism argued (brief §7.2)	Section 4 (Augmentation), Figure 3
Model name, variant, pretrained weights; input size (brief §7.3)	Table 3
Batch size, gradient accumulation; epochs, early stopping (brief §7.3)	Table 3
Optimiser, learning rate, schedule; weight decay, loss (brief §7.3)	Table 3, Table B.4
Class weights or sampling; random seed (brief §7.3)	Table 3, Section 2.4
Software framework and library versions; hardware, GPU memory (brief §7.3)	Section 4.4, Table B.1
Fairness of the comparison (rubric)	Section 4.4 (paragraph “Fairness”), Table 3
Results and evaluation (4 marks)	
Test accuracy; precision, recall, F1; macro-F1 (brief §8)	Table 4
Per-class precision, recall and F1 (brief §8)	Figure 5; Table A.1, Table A.2 (Appendix A)
Confusion matrix (brief §8)	Figure 6 (all three models, shared colour scale)
Training and validation loss curves (brief §8)	Figure 4 (top row), Table B.2
Validation accuracy or F1 curve (brief §8)	Figure 4 (bottom row)
Several correct and incorrect predictions (brief §8)	Figure 7, Figure 9
Comparison of all three models (rubric)	Table 4, Table 5, Figure 8, Table B.3
Discussion, error analysis and conclusion (2 marks)	
Important classification errors (brief §8)	Section 6.2, Table 6
Effects of class imbalance and data quality (brief §8)	Section 6.2, Section 2.4
Plausible reasons for performance differences (brief §8)	Section 6.1
Limitations (brief §9)	Section 6.5
Justified conclusion and recommended model (rubric)	Section 7 (incl. lessons learned)
Report quality and references (1 mark)	
Nine-part structure, 10–15 pages, PDF (brief §9)	title page with group members and abstract; sections 1–7; references
Sources, pretrained models, software acknowledged (brief §9)	Section 2.1, Section 3, Table B.1, references
Submission items (brief §11)	
Report as PDF	`docs/itri626_report/main.pdf`
Presentation slides	to be prepared by the group (not part of this repository)
Notebook with visible outputs, figures, metric tables	`notebooks/ITRI626_four_models.ipynb`
Python scripts for data preparation and evaluation	`tools/build_dataset_v2.py`, `tools/export_splits.py`, `scripts/itri626_experiments.py`, `src/compclass/`
README with dataset source, setup, reproduction steps	`README.md`, section “ITRI626 submission”
Split files or a script that recreates them	`data/v2/splits/`, `tools/export_splits.py`
Repository link or archive; data access instructions	README “ITRI626 submission”; Appendix D